Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Software Development For Information Systems Project

Dimitrios Nikolaos Boutzounis - Stavrou Spyridon - Iasonas Stephanou

January 2026

Contents

1 Overview **2**

2 Baseline **2**
2.1 Performance & Statistics 2

3 Late Materialization **2**
3.1 Implementation Details 2
3.2 Performance & Statistics 3

4 Column Store **3**
4.1 Implementation Details 3
4.2 Performance & Statistics 4

5 Unchained Hash Table **5**
5.1 Implementation Details 5
5.2 Performance & Statistics 5

6 Index Optimization **6**
6.1 Implementation Details 6
6.2 Performance & Statistics 6

7 Parallel Build **7**
7.1 Implementation Details 7
7.2 Performance & Statistics 7

8 Parallel Probe **8**
8.1 Implementation Details 8
8.2 Performance & Statistics 8

9 Summarized Results **9**

1 Overview

The goal of this project is to optimize a high-performance, in-memory join pipeline executor from the SIGMOD 2025 programming contest. The system must be capable of executing complex pipelines of join operations as quickly as possible. The primary focus is on implementing low-level optimizations to minimize end-to-end execution time for heavy analytical joins.

First we focused on optimizing memory access and execution speed. We needed to transition the engine's intermediate storage from row-store to a cache-friendly column-store format. To reduce data movement, we implemented late materialization, which uses lightweight 64-bit references for strings instead of copying the actual data. Additionally, we implemented Unchained Hashtable with Bloom filters, a specialized structure that uses a contiguous memory buffer to make join lookups significantly faster than traditional approaches.

Then the focus was on scaling performance through multi-threading. We implemented direct indexing for integer columns without null values, allowing the engine to process raw input pages without any redundant copying. The join process was fully parallelized using a partition-and-build strategy, supported by work stealing to dynamically balance the load between threads. To prevent bottlenecks during this parallel execution, we also build a custom slab allocator to handle memory allocations efficiently without system contention.

2 Baseline

2.1 Performance & Statistics

The following table showcases the performance and statistics of the Baseline solution from the the SIGMOD CONTEST 2025, which uses the default STL Unordered Map to store the keys and the row id's used in the join pipeline.

Table 1: Baseline (Averaged over 3 Runs)

Metric	**Value**	**Unit**	**Notes**
Total Runtime	142,842	ms	Avg. of 3 runs (Min: 141.4s, Max: 144.5s)
Peak Memory (RSS)	9.34	GB	Max Resident Set Size
Avg. CPU Usage	106.33	%	Avg. of 3 CPU usages (Min: 105%, 107%)
Machine CPU	Apple M1 Pro	-	-

3 Late Materialization

3.1 Implementation Details

The materialization introduces a compact and efficient way to represent and recover string (VARCHAR) values during query execution. Smart_string encodes

the location of a VARCHAR inside the original column-store input using a single 64-bit value. The encoding stores: table ID, column ID, page ID and offset index (position of the string within the page). This avoids copying string data during intermediate execution. The actual string is reconstructed only when needed through get_value(), which:

- handles regular pages with offsets
- correctly assembles multi-page long strings (0xffff / 0xfffe markers)

Value_t is a lightweight tagged union for execution It stores INT32 or SMART_STRING using bit-packed metadata, uses two low bits to encode type and supports null representation without variant overhead.

Together, Smart_string and value_t enable late materialization, minimizing string copies and improving performance throughout the execution engine.

3.2 Performance & Statistics

Table 2: Late Materialization (Averaged over 3 Runs)

Metric	**Value**	**Unit**	**Notes**
Total Runtime	96,699	ms	Avg. of 3 runs (Min: 96.1s, Max: 97.4s)
Peak Memory (RSS)	8.47	GB	Max Resident Set Size
Avg. CPU Usage	97.66	%	Avg. of 3 runs (Min: 96%, Max: 99%)
Machine CPU	Apple M1 Pro	-	-

Comparing the results to the Baseline implementation (Table 1), Late Materialization achieved a significant reduction in execution time, dropping from 142,842 ms to 96,699 ms. This represents a performance improvement of approximately 32%. Additionally, peak memory usage decreased by nearly 1GB. These gains confirm the effectiveness of deferring VARCHAR materialization, as it eliminates the redundant copying of heavy string data during intermediate join steps, effectively reducing memory bandwidth pressure.

4 Column Store

4.1 Implementation Details

This optimization task replaces the intermediate row-store representation with a paginated column-store layout to improve cache locality and join performance.

The Column_t structure stores values column-by-column across fixed-size pages:

- Paged Storage: Each column consists of multiple Page_t blocks. The page index and offset inside the page are computed via bit shifting (PAGE_SHIFT, PAGE_MASK), enabling fast indexing.

- Efficient Appends: push_back() appends values sequentially. When a page is full, a new Page_t is allocated. This reduces allocation overhead in tight join loops.
- Fast Random Access: get_at() computes the correct page and offset to retrieve any value in O(1) time. Out-of-range accesses safely return NULL.
- Predictable Memory Layout: Values of the same column are stored contiguously, improving cache usage and preparing the executor for parallel join processing in later assignments.

This columnar format becomes the backbone for intermediate results produced by scans and joins, replacing all row-store structures in the execution pipeline.

4.2 Performance & Statistics

Table 3: Column Store (Averaged over 3 Runs)

Metric	**Value**	**Unit**	**Notes**
Total Runtime	47,546	ms	Avg. of 3 runs (Min: 47.3s, Max: 47.7s)
Peak Memory (RSS)	6.62	GB	Max Resident Set Size
Avg. CPU Usage	106.66	%	Avg. of 3 runs (Min: 103%, Max: 109%)
Machine CPU	Apple M1 Pro	-	-

Transitioning to a Column Store format for intermediate results yielded a massive performance boost compared to the Late Materialization phase (Table 2). The average execution time dropped from 96,699 ms to 47,546 ms. This represents a 2x speedup (approx 50% reduction in runtime), proving that processing data in a columnar layout is far more efficient for this workload.

We also observed a substantial drop in memory usage, which fell from 8.47 GB to 6.62 GB. This improvement happens because the column-store format improves CPU cache locality, as the processor can read relevant data sequentially without "jumping" around in memory. By keeping related data packed together, we maximize cache hits and significantly reduce the memory bandwidth required for complex joins.

5 Unchained Hash Table

5.1 Implementation Details

The Unchained Hash Table implementation introduces a high-performance, cache-conscious structure optimized for join processing. This design utilizes a contiguous buffer storage model (adjacency array layout). All (key, row_id) pairs are stored in a single linear array, grouped physically by their hash prefix. This layout ensures that tuples sharing the same prefix are stored adjacently in memory, significantly improving spatial locality and cache efficiency during scan operations.

To support this layout, we need a hybrid directory structure that acts as both an index and a filter. The directory is a vector of 64-bit entries, where each entry is split to maximize utility: the upper 48 bits store the cumulative count (or offset) of keys up to that slot, defining the exact memory range for the bucket, while the lower 16 bits store a Bloom-style tag. High-quality distribution is achieved using hardware-accelerated hashing (crc32), where the upper bits of the resulting 64-bit hash determine the directory slot and the lower bits select the specific tag.

A key optimization in this implementation is the use of Bloom-style tags for early filtering. To maintain a low false-positive rate, the system uses a precomputed table of tags (tags[2048]), each containing a bitmask with exactly four bits set to 1. During the build phase, the tag corresponding to a key's hash is logically OR'ed into the directory entry's lower bits. During the probe phase, we perform a fast bitwise check ($!(tag\& \sim entry_tag)$). This allows the lookup routine to instantly reject non-matching keys before accessing the main tuple buffer.

The Build Phase begins with counting the frequency of each hash prefix to determine the necessary storage size. This is followed by allocating the single contiguous buffer and computing prefix sums to set the exact boundaries for each bucket. Finally, we place every tuple into its pre-calculated position and OR the tag with the lower 16 bits of the directory slot.

5.2 Performance & Statistics

For this experiment, the Unchained Hash Table was configured with a shift parameter of 48, which was our best compared to the other values we tried.

Table 4: Unchained Hash Table (Averaged over 3 Runs)

Metric	**Value**	**Unit**	**Notes**
Total Runtime	38,668	ms	Avg. of 3 runs (Min: 38.5s, Max: 38.8s)
Peak Memory (RSS)	6.52	GB	Max Resident Set Size
Avg. CPU Usage	113.33	%	Avg. of 3 runs (Min: 111%, Max: 115%)
Machine CPU	Apple M1 Pro	-	-

Replacing the standard STL Unordered Map (used in the Column Store step) with our custom Unchained Hash Table improved performance significantly. The total execution time dropped from 47,546 ms to 38,668 ms, an improvement of roughly 19%.

This speedup happens because the Unchained design stores data in a single continuous block of memory. By keeping data close together and using Bloom filters to quickly reject non-matches, the processor can access data much faster than it can with the scattered memory layout of the previous solution.

6 Index Optimization

6.1 Implementation Details

To save memory and speed up processing, we upgraded the Column_t class so it can index data directly from the input pages without making unnecessary copies. We designed Column_t to act as a smart wrapper that automatically picks the best way to store data. If an integer column has no empty (null) values, it uses a special "zero-copy" mode that simply points to the original data instead of duplicating it. For other data types, it continues to copy and manage memory as usual.

This approach keeps the rest of the system fast and simple. Because the commands to read data (like get_at) stay exactly the same, the rest of the database engine doesn't need to know or care how the data is stored behind the scenes. This means we don't have to add slow checks or complicated logic to the query execution code. The column handles all the specific details internally, giving us the performance boost of direct memory access without risking errors or breaking existing features.

6.2 Performance & Statistics

Table 5: Index Optimization (Averaged over 3 Runs)

Metric	**Value**	**Unit**	**Notes**
Total Runtime	26,470	ms	Avg. of 3 runs (Min: 26.4s, Max: 26.6s)
Peak Memory (RSS)	5.90	GB	Max Resident Set Size
Avg. CPU Usage	84.66	%	Avg. of 3 runs (Min: 82%, Max: 87%)
Machine CPU	Apple M1 Pro	-	-

The implementation of index optimizations specifically avoiding unnecessary data copying for INT32 columns without null values delivered a substantial performance improvement.

- Runtime Reduction: The total runtime dropped significantly from 38,668 ms to 26,470 ms, resulting in a 1.46x speedup (a 31.55% reduction in total

execution time). This confirms that memory allocation and copying were major problems in the initial implementation.

- Memory Efficiency: Peak resident memory usage decreased from 6.52 GB to 5.90 GB (a $\approx$ 9.5% reduction). By indexing directly on the input pages instead of creating deep copies for dense integer columns, the system reduced memory pressure and allocation overhead.
- CPU Usage: Average CPU usage decreased from 113.33% to 84.66%. This reduction indicates that the baseline implementation spent a significant amount of CPU cycles on redundant memory operations (allocation and copying), which have now been eliminated.

7 Parallel Build

7.1 Implementation Details

To speed up the creation of the hash table, we divide the work among multiple threads that operate independently using a "divide and conquer" strategy. We implemented a custom "slab allocator" with three levels (Global, Thread, and Partition). This allows every thread to grab large chunks of memory upfront and organize data into local buffers without needing expensive locks or frequent system calls.

Once all threads finish collecting their data, the system performs a quick synchronization step to calculate exactly how much space is needed for the final table. We then allocate one single, continuous block of memory to hold the result. The final construction happens in two fast passes for each partition. First, the main thread counts exactly where each item belongs to ensure they are packed tightly together in memory . Second, to fill the whole contiguous buffer, we use a dynamic load-balancing strategy where threads automatically "steal" the next available partition to process.

7.2 Performance & Statistics

All the runs executed with the maximum possible threads of the CPU (8 threads) and the partitions were $4 * num_threads$.

Table 6: Parallel Build (Averaged over 3 Runs)

Metric	**Value**	**Unit**	**Notes**
Total Runtime	26,081	ms	Avg. of 3 runs (Min: 25.9s, Max: 26.1s)
Peak Memory (RSS)	5.96	GB	Max Resident Set Size
Avg. CPU Usage	87.00	%	Avg. of 3 runs (Min: 84%, Max: 90%)
Machine CPU	Apple M1 Pro	-	-

The transition to a parallelized build phase resulted in a modest incremental improvement, serving primarily as a structural foundation for subsequent optimizations rather than an immediate performance leap.

- Runtime Reduction: The total runtime decreased from 26,470 ms to 26,081 ms, yielding a 1.015x speedup (approx. 1.5% reduction). This indicates that for this workload, the hash table construction (the "build" side) accounts for a relatively small fraction of the total execution time compared to the probing phase.
- CPU Utilization: Average CPU usage increased slightly from 84.66% to 87.00%. This reflects the activation of multiple threads during the build phase. However, because the computationally expensive probe phase remained serial at this stage, the overall CPU utilization did not saturate the available cores.
- Memory Stability: Peak memory usage remained remarkably stable, rising only marginally from 5.90 GB to 5.96 GB. This demonstrates the efficiency of the slab allocator design, which manages thread-local buffers without incurring significant fragmentation.

8 Parallel Probe

8.1 Implementation Details

To maximize performance during the probing phase, we implemented a dynamic work-stealing strategy. Instead of pre-assigning a fixed portion of the data to each thread, we use a shared atomic counter to distribute work in small "chunks" on demand. This ensures that as soon as a thread finishes its current batch of tuples, it immediately claims the next available range. This approach automatically balances the load across all processor cores, preventing faster threads from idling while waiting for slower ones to finish.

Threads write their matches exclusively to their local memory, avoiding expensive contention. Once all probing is complete, a final single-threaded step efficiently merges these isolated buffers into the final output, ensuring the result is both deterministic and produced as quickly as possible.

8.2 Performance & Statistics

All the runs executed with the maximum possible threads of the CPU (8 threads) and the partitions were $4 * num_threads$.

Table 7: Parallel Probe (Averaged over 3 Runs)

Metric	Value	Unit	Notes
Total Runtime	11,242	ms	Avg. of 3 runs (Min: 10.9s, Max: 11.4s)
Peak Memory (RSS)	6.14	GB	Max Resident Set Size
Avg. CPU Usage	166.00	%	Avg. of 3 runs (Min: 163%, Max: 170%)
Machine CPU	Apple M1 Pro	-	-

The transition from parallelizing the build phase to parallelizing the probe phase resulted in the most significant performance leap for the join operation.

- Runtime Reduction: The total runtime dropped from 26,081 ms to 11,242 ms, representing a 2.32x speedup (approximately 57% reduction in execution time).
- CPU Utilization: The average CPU usage nearly doubled, jumping from 87% to 166%. This confirms that the probing phase is the dominant component of the workload. While the "Parallel Build" configuration was combined with a serial probe phase (keeping average CPU usage below 100%), enabling Parallel Probing allowed the application to effectively utilize the multi-core architecture of the Apple M1 Pro.
- Memory Overhead: Peak memory usage increased slightly from 5.96 GB to 6.14 GB (+3%). This minor overhead is an expected trade-off for the thread-local storage and buffers required to sustain parallel execution, but it remains well within efficient limits given the massive performance gain.

The current implementation checkpoint produced the fastest execution time on the given queries (1a - 33c) from the IMDB dataset and was chosen as the final and optimum submission for the project.

9 Summarized Results

Table 8: Optimization Pipeline Performance Summary

Implementation Stage	Runtime (ms)	Total Time Reduction	Incremental Reduction
Baseline	142,842	0.00%	0.00%
Late Materialization	96,699	32.30%	32.30%
Column Store	47,546	66.71%	50.83%
Unchained Hash Table	38,668	72.93%	18.67%
Index Optimization	26,470	81.47%	31.55%
Parallel Building	26,081	81.74%	1.47%
Parallel Probing	11,242	92.13%	56.90%